NORTH BENGAL INTERNATIONAL UNIVERSITY

Project Title: Doctor Appointment System

PRESENTED TO:

Rifa Zumana

Lecturer

Department of CSE, NBIU

PRESENTED BY:

MST Sumaya khatun

ID: 0712310012101039

Department of CSE, NBIU

Submission date: 07/5/2026

Statement of Ordinary

I hereby declare that this report, titled “Doctor Appointment System”, is the result of our own work. All information, data, and sources used in this report have been appropriately cited and referenced in accordance with academic and ethical standards.

I confirm that the content presented in this report is original and has not been submitted, in part or in full, for any other academic or professional qualification. Contributions from other individuals or sources have been duly acknowledged.

--- ---

Students signature Supervisor signature

Date: Date:

Table Of Contents

Statement of Originality 1

Introduction…….. 3

Objective..... 4

Motivation……..... 5

Literature review……. 6

Methodology 7

System Analysis…..... 8

Software Requirements 9

System Architecture 10

Technology Stack 11

Backend Code Structure …..... 12

Frontend Code Structure 13

Database Description 14,15,16

API Description……..... 17

User Interface 18,19,20,21

Features 22

Implementation and Testing 23

Challenge and Solution 24

Future Work 25

Conclusion..... 26

References 27

1. Introduction

The Doctor Appointment System is a modern web-based application designed to simplify and automate the process of scheduling medical appointments. In traditional healthcare systems, patients often face difficulties such as long waiting times, lack of proper scheduling, and inefficient communication with doctors. These problems not only waste time but also reduce the overall quality of healthcare services.

With the rapid advancement of technology, digital solutions have become essential in improving healthcare management systems. This project aims to provide an efficient, user-friendly, and reliable platform where patients can easily book appointments with doctors from anywhere at any time. At the same time, doctors can manage their schedules, review appointments, and update patient statuses effectively.

The system is developed using the MERN stack, which includes MongoDB for database management, Express.js and Node.js for backend development, and React.js for frontend user interface design. This combination ensures high performance, scalability, and smooth user experience.

Additionally, the system incorporates secure authentication using JSON Web Tokens (JWT) and provides role-based access control for different types of users such as admin, doctor, and patient. An integrated email notification system is also included to notify users about appointment confirmations and updates.

Overall, this project demonstrates how modern web technologies can be used to solve real-world healthcare problems by creating a structured, efficient, and accessible appointment management system.

2. Objective

- The primary objective of the Doctor Appointment System is to develop a reliable and efficient web-based platform that simplifies the process of scheduling medical appointments between patients and doctors. The system aims to eliminate the limitations of traditional manual booking methods by introducing a digital solution that ensures accuracy, accessibility, and convenience.

- One of the key objectives of this project is to provide a user-friendly interface where patients can easily browse available doctors, select suitable time slots, and book appointments without any physical effort. This reduces waiting time and improves overall patient experience.

- Another important objective is to enable doctors to manage their schedules effectively. Doctors can view appointment requests, approve or reject bookings, and update the status of appointments. This helps in maintaining an organized workflow and avoiding scheduling conflicts.

- The system also aims to ensure secure user authentication and authorization through the implementation of JSON Web Tokens (JWT). Role-based access control is introduced to differentiate functionalities among admin, doctor, and patient users, ensuring system security and proper data handling.

- Additionally, the project focuses on implementing an automated email notification system. This feature keeps users informed about appointment confirmations, updates, and status changes, thereby improving communication between patients and doctors.

- Finally, the system is designed to be scalable and maintainable using modern technologies such as the MERN stack. This ensures that the application can be further enhanced in the future by adding new features such as online payments, video consultations, and mobile application support.

3. Motivation

The motivation behind developing the Doctor Appointment System arises from the common challenges faced in traditional healthcare appointment processes. In many hospitals and clinics, appointment scheduling is still handled manually, which often leads to long waiting times, mismanagement of schedules, and lack of proper communication between patients and doctors. These inefficiencies create frustration for patients and increase the workload for healthcare providers.

With the increasing use of the internet and digital technologies, there is a strong need to modernize healthcare systems. Patients today expect quick and convenient services that allow them to access healthcare facilities without unnecessary delays. This project is motivated by the idea of leveraging modern web technologies to create a seamless and efficient appointment booking experience.

Another key motivation is to reduce human errors in scheduling. Manual systems are prone to mistakes such as double booking, lost records, or incorrect information. By automating the process, the system ensures accurate data management and minimizes the chances of such errors.

The project is also inspired by the need to improve communication between doctors and patients. Often, patients are not informed about changes in appointment status, which leads to confusion and inconvenience. By integrating an email notification system, the application ensures that users receive timely updates regarding their appointments.

Furthermore, as a computer science student, this project provides an opportunity to apply theoretical knowledge in a practical environment. It allows the use of modern technologies like React, Node.js, Express, and MongoDB to build a real-world application. This not only enhances technical skills but also helps in understanding how to design scalable and user-centric systems.

In summary, the motivation of this project is to bridge the gap between traditional healthcare systems and modern digital solutions by creating a smart, efficient, and user-friendly appointment management platform.

4. Literature Review / Background Study

The development of online appointment management systems has gained significant attention in recent years due to the increasing demand for efficient healthcare services. Various researchers and developers have proposed different solutions to improve the traditional appointment booking process using web-based and mobile technologies.

Several existing systems focus on digitizing appointment scheduling by allowing patients to book appointments online. These systems aim to reduce waiting time and improve accessibility. For example, many hospital management systems provide basic features such as doctor listing, appointment booking, and schedule management. However, a large number of these systems lack advanced functionalities like real-time notifications, secure authentication, and user-friendly interfaces.

Research studies have shown that integrating web technologies into healthcare systems can significantly improve operational efficiency. Modern applications use frameworks like React for dynamic user interfaces and Node.js for scalable backend services. These technologies enable fast performance and better user experience compared to traditional desktop-based systems.

Some existing platforms also incorporate notification systems such as SMS or email alerts to inform users about appointment confirmations and updates. However, many of these implementations are either costly or complex, making them less accessible for smaller healthcare providers. This project addresses that gap by implementing a simple yet effective email notification system using Nodemailer.

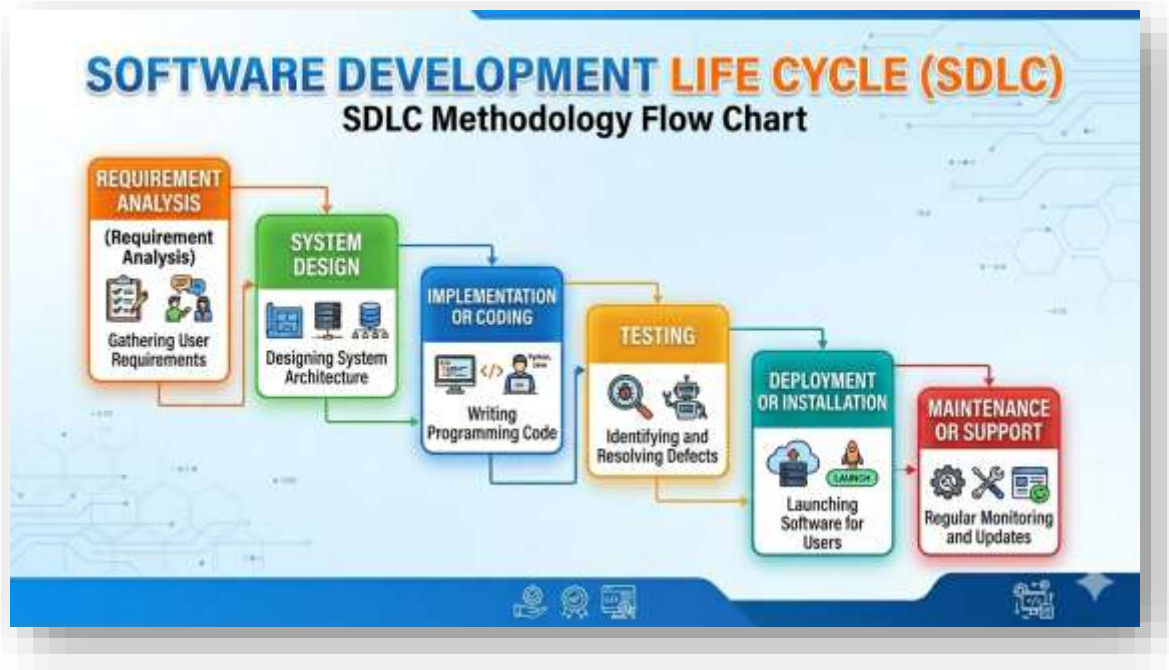
Security is another important aspect discussed in previous works. Many systems emphasize the use of authentication mechanisms to protect user data. Technologies like JSON Web Tokens (JWT) and password hashing techniques are widely adopted to ensure secure access and data privacy.

Despite the availability of various solutions, there is still a need for a lightweight, scalable, and easy-to-use system that combines all essential features in one platform. The Doctor Appointment System developed in this project attempts to integrate these functionalities, including secure login, role-based access, appointment management, and automated notifications, into a single cohesive system.

5. Methodology

The development of the Doctor Appointment System followed the Agile Software Development Life Cycle (SDLC) to ensure flexibility and continuous improvement.

1. **Requirement Analysis:** Identifying the essential features needed for an EMS, such as employee profiles and login portals.
2. **System Design:** Drafting the architecture of the application and the component structure in React.
3. **Database Modeling:** Designing the MongoDB schema to store employee objects efficiently.
4. **Development:** Building the UI with React and the API with Node.js/Express.
5. **Testing:** Performing unit testing on API routes and UI testing to ensure responsiveness.
6. **Deployment:** Hosting the application on cloud platforms.



6. System Analysis (Software Requirements Specification - SRS)

The System Analysis phase is a critical part of the Software Development Life Cycle (SDLC) that defines the requirements and constraints of the project. The Software Requirements Specification (SRS) document provides a detailed description of the hardware and software needed to develop, deploy, and run the Doctor Appointment System (DAS) effectively.

6.1 Hardware Requirements

The hardware requirements specify the physical resources needed for both the development environment and the end-user environment. Since the Doctor Appointment System is a web-based application, it does not require high-end hardware for the end-user, but the development environment should be robust.

A. Development Environment (Minimum):

- Processor: Dual-core processor (Intel i5 or AMD Ryzen 5) or higher.
- RAM: Minimum 8 GB (Recommended 16 GB for smooth multitasking with VS Code, Browser, and Node.js).
- Storage: 256 GB SSD (Solid State Drive) is preferred for faster compilation and execution.
- Monitor: Standard HD Display (1366x768) or higher for UI/UX designing.
- Internet: Stable connection for downloading dependencies (NPM packages) and cloud database connectivity.

B. Client/User Environment (End-User):

- Processor: Basic dual-core processor.
- RAM: 2 GB or higher.
- Device: Any device with a modern web browser (Laptop, Desktop, Tablet, or Smartphone).

6.2 Software Requirements

The software requirements define the tools, languages, and platforms used to build and host the system.

A. Development Tools:

- Operating System: Windows 10/11, macOS, or Linux (Ubuntu).
- Integrated Development Environment (IDE): Visual Studio Code (VS Code) for writing and debugging code.
- Version Control: Git and GitHub for managing code versions and collaboration.

B. Technology Stack (MERN Stack):

- Frontend: React.js (v18+) for building a dynamic and responsive user interface.
- Backend: Node.js as the runtime environment and Express.js as the web framework.
- Database: MongoDB Atlas (Cloud-based NoSQL database) for storing employee records.
- Authentication: Firebase Authentication for secure login and account management.

C. Client-Side Software:

- Web Browser: Any modern web browser such as Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari.

7.System Architecture

The Doctor Appointment System is designed using a Client-Server Architecture, which ensures scalability, maintainability, and efficient communication between different components of the system. The architecture is divided into three main layers: Presentation Layer, Application Layer, and Data Layer.

The system follows a **Client-Server Architecture**. It is divided into three distinct layers:

1.Presentation Layer (Frontend): The Presentation Layer is responsible for handling all user interactions. It is developed using **React.js**, which provides a dynamic and responsive user interface.

- Patients can register, log in, and book appointments
- Doctors can view and manage appointment requests
- Admins can monitor system activities
- The frontend communicates with the backend using RESTful APIs and handles data rendering based on user actions. It also manages state and ensures a smooth user experience.

1. **Logic Layer (Backend):** The Node.js and Express server acts as the bridge. It receives requests from the frontend and communicates with the database.
2. **Data Layer (Database):** MongoDB stores all employee records in JSON-like documents.
3. **Authentication Layer**: Firebase Authentication manages secure user login, registration, and session handling, ensuring only authorized users can access the system.
4. **API Layer**: RESTful APIs are used to enable communication between the frontend and backend. These APIs handle CRUD operations using HTTP methods like GET, POST, PUT, and DELETE.

8. Technology Stack

The project utilizes the **MERN** stack along with **Firebase** for a robust experience:

- React: Used for building a dynamic, single-page application (SPA) with reusable components and efficient state management.

- Node.js: A JavaScript runtime that enables server-side execution and handles asynchronous operations efficiently.
- Express.js: A lightweight framework for Node.js that simplifies API development and routing.

- MongoDB: A NoSQL database used to store and manage employee data in a flexible and scalable way.
- Firebase Authentication: Handles secure user authentication, including login, registration, and session management.

- Tailwind CSS: A utility-first CSS framework used to design a responsive, modern, and clean user interface.
- React Router: Used for client-side routing and navigation between different pages without reloading the application.

- Axios: Used to send HTTP requests from the frontend to the backend APIs.
- CORS: Enables secure communication between frontend and backend hosted on different origins.

9.Backend Code Structure:

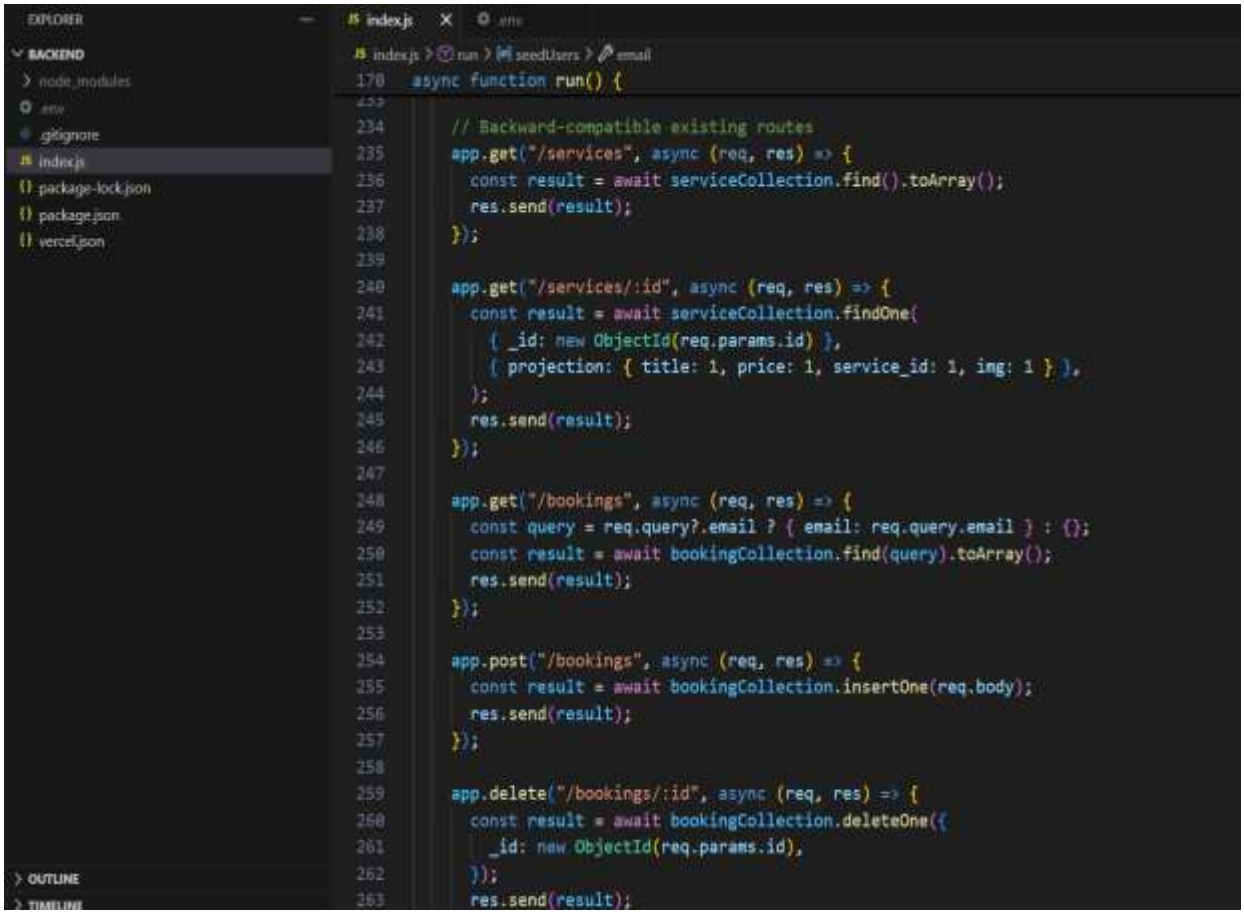


Fig: backend code structure

10.Frontend Code Structure:

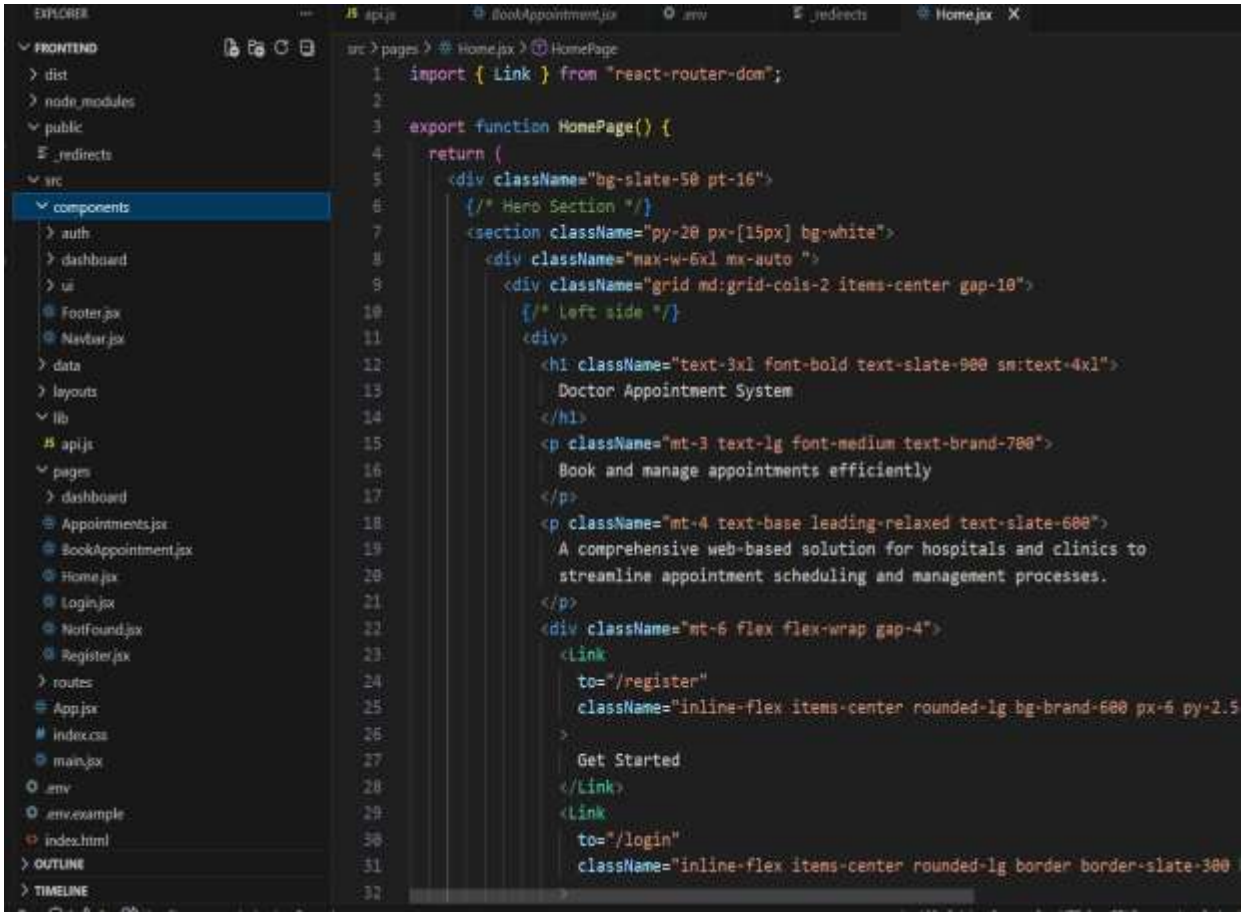


Fig: frontend code structure

11. Database Description

The Doctor Appointment System uses MongoDB, a NoSQL database, to store and manage all application data efficiently. MongoDB is chosen for its flexibility, scalability, and ability to handle large amounts of unstructured data. The database is hosted on MongoDB Atlas, which provides cloud-based storage with high availability and security.

1. Database Structure Overview

The system follows a **collection-based structure**, where data is stored in separate collections instead of traditional tables. The main collections used in this project are:

- Users Collection
- Appointments Collection
- Services Collection (optional, based on implementation)

Each collection stores related data in the form of JSON-like documents.

2. Users Collection:

The Users collection stores information about all types of users in the system, including patients, doctors, and administrators.

Key Fields:

- _id: Unique identifier
- name: User's full name
- email: User's email (unique)
- password: Encrypted password
- role: Defines user type (admin / doctor / patient)
- createdAt: Account creation timestamp

```
_id: ObjectId('69d8c2bfd30f09d492cd1712')
name : "Admin User"
email : "admin@demo.com"
role : "admin"
specialization : "General"
passwordHash : "$2b$10$WfxYisXCot3X7RvUmYTKx.rjpo
createdAt : "2026-04-10T09:28:31.342Z"
updatedAt : "2026-04-10T09:28:31.342Z"
```


Fig: Users Collection

3. Appointments Collection

The Appointments collection is the core of the system, storing all appointment-related data.

Key Fields:

- _id: Unique appointment ID
- patientId: Reference to patient (User ID)
- doctorId: Reference to doctor (User ID)
- appointmentDate: Selected date and time
- status: (Pending / Approved / Completed / Cancelled)
- notes: Optional message or reason
- createdAt: Timestamp

```
_id: ObjectId('69d8c325d30f09d492cd1717')
patientId : "69d8c311d30f09d492cd1716"
patientName : "Jahid"
patientEmail : "jahid20cse37@gmail.com"
doctorId : "69d8c2bfd30f09d492cd1714"
doctorName : "Dr. Hamza Ali"
date : "2026-04-18"
status : "Pending"
notes : ""
createdAt : "2026-04-10T09:30:13.257Z"
updatedAt : "2026-04-10T09:30:13.257Z"
```


Fig: Appointments collection

4.Data Flow in Database

1. User registers → data stored in Users collection
2. Patient books appointment → stored in Appointments collection
3. Doctor updates status → database is updated
4. Email notification triggered based on status change

12. API Description

The backend of the Doctor Appointment System is implemented as a RESTful API, which enables communication between the frontend and the database. The API is built using Node.js and Express.js, and it handles all core functionalities such as authentication, user management, appointment booking, and status updates.

Each API endpoint follows standard HTTP methods like GET, POST, PUT/PATCH, and DELETE to perform specific operations.

1. Authentication APIs

These endpoints manage user registration and login:

- **POST /api/auth/register**
  Registers a new user (patient, doctor, or admin)
- **POST /api/auth/login**
  Authenticates user and returns a JWT token

2. User Management APIs

- **GET /api/users**
  Fetches all users (Admin only)
- **GET /api/users/:id**
  Fetches a specific user's details

3. Appointment APIs

These are the core APIs for managing appointments:

- **GET /api/appointments**
  Fetches all appointments (role-based filtering applied)
- **POST /api/appointments**
  Creates a new appointment (Patient action)
- **PATCH /api/appointments/:id/status**
  Updates appointment status (Doctor action: Approve / Complete / Cancel)
- **DELETE /api/appointments/:id**
  Cancels or removes an appointment

13. User Interface (UI)

The UI is designed with a focus on professional aesthetics and usability.

- **Dashboard:** A high-level summary showing total employee counts.
- **Employee Table:** A clean table that lists all employees with edit/delete buttons.
- **Forms:** Validated forms for adding or editing employee details.

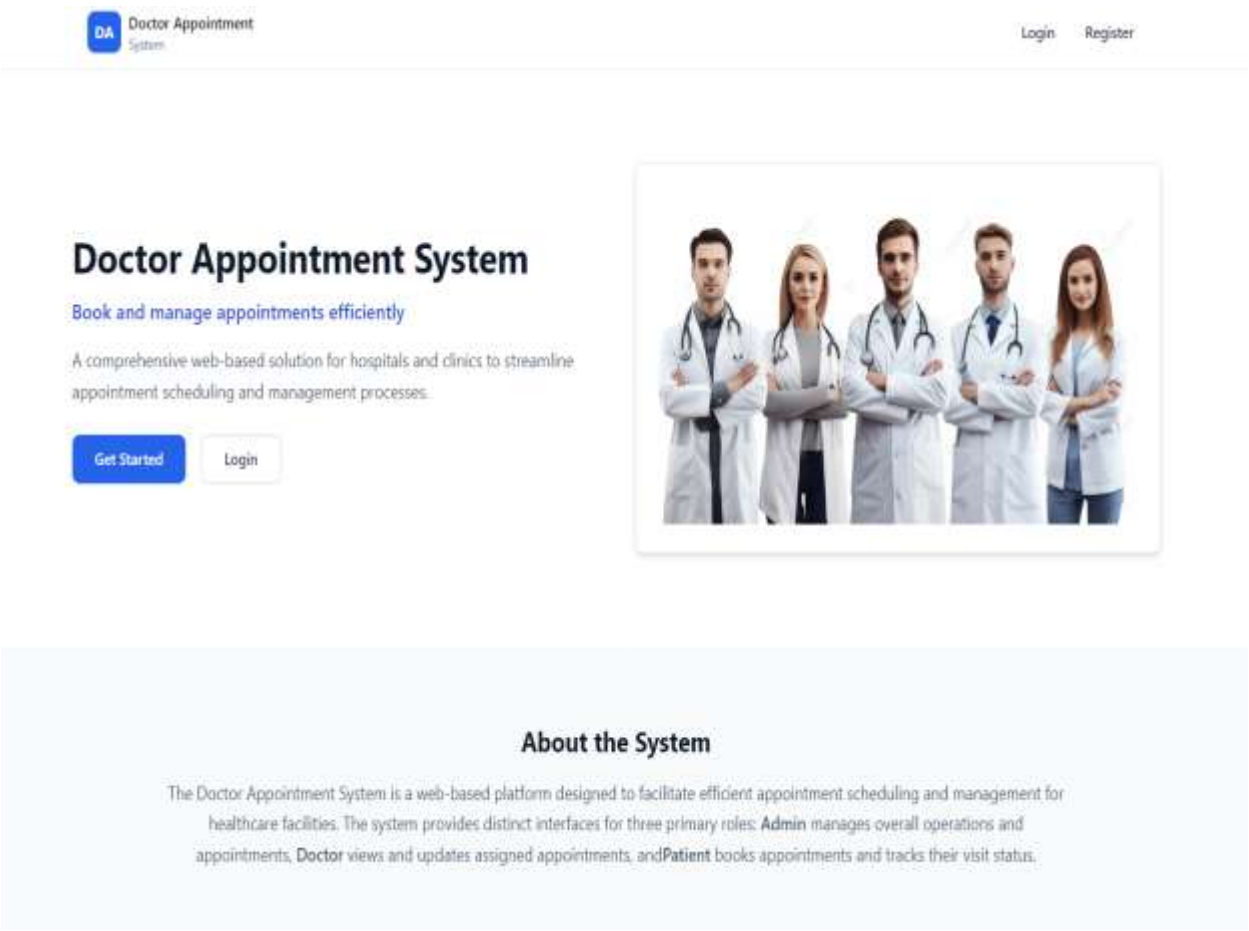


Fig: Home Page

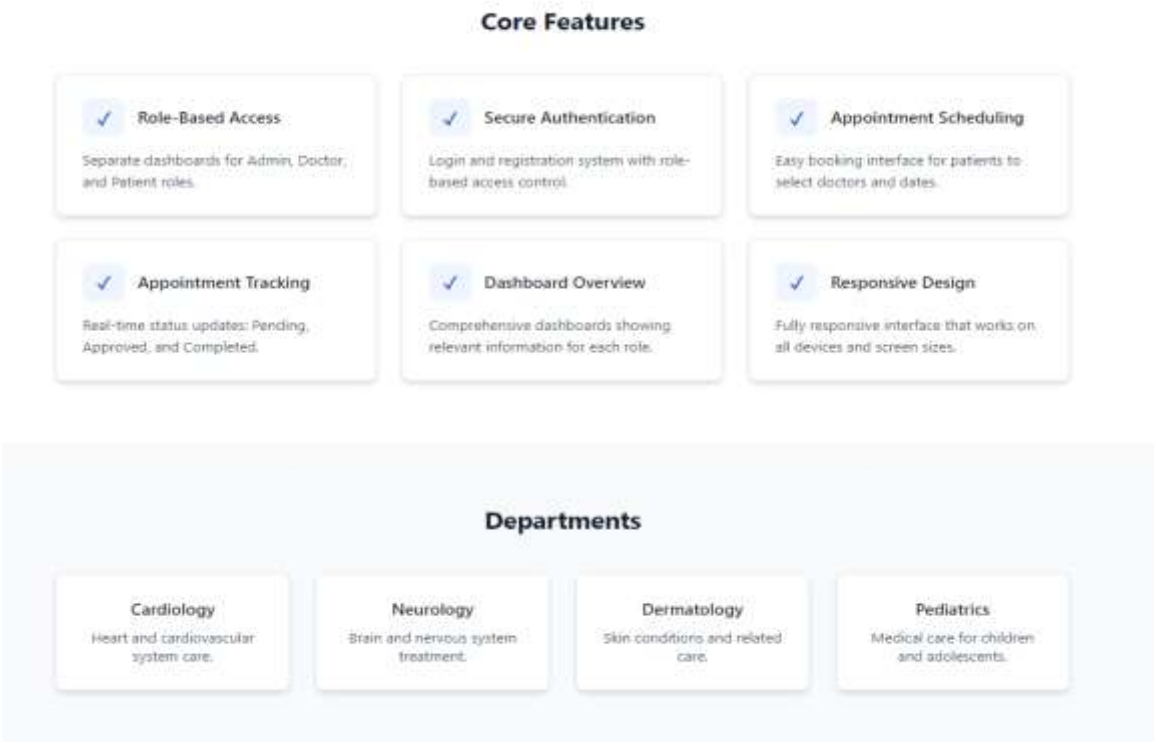


Fig: all feature and department

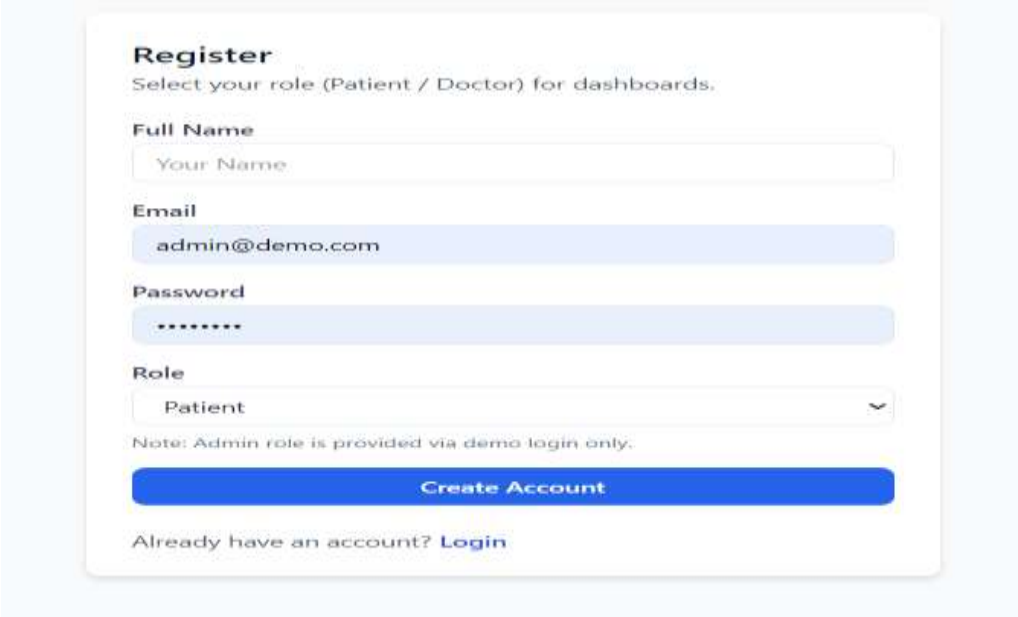


Fig: Register Page

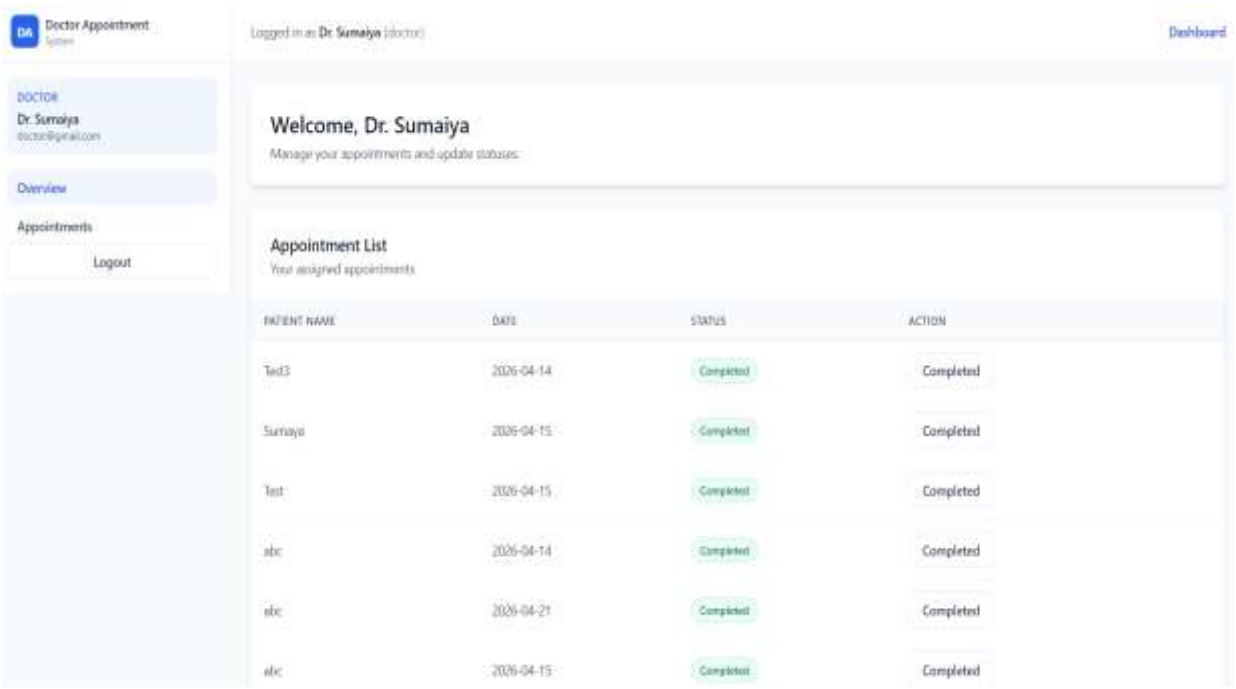


Fig: Doctor Dashboard

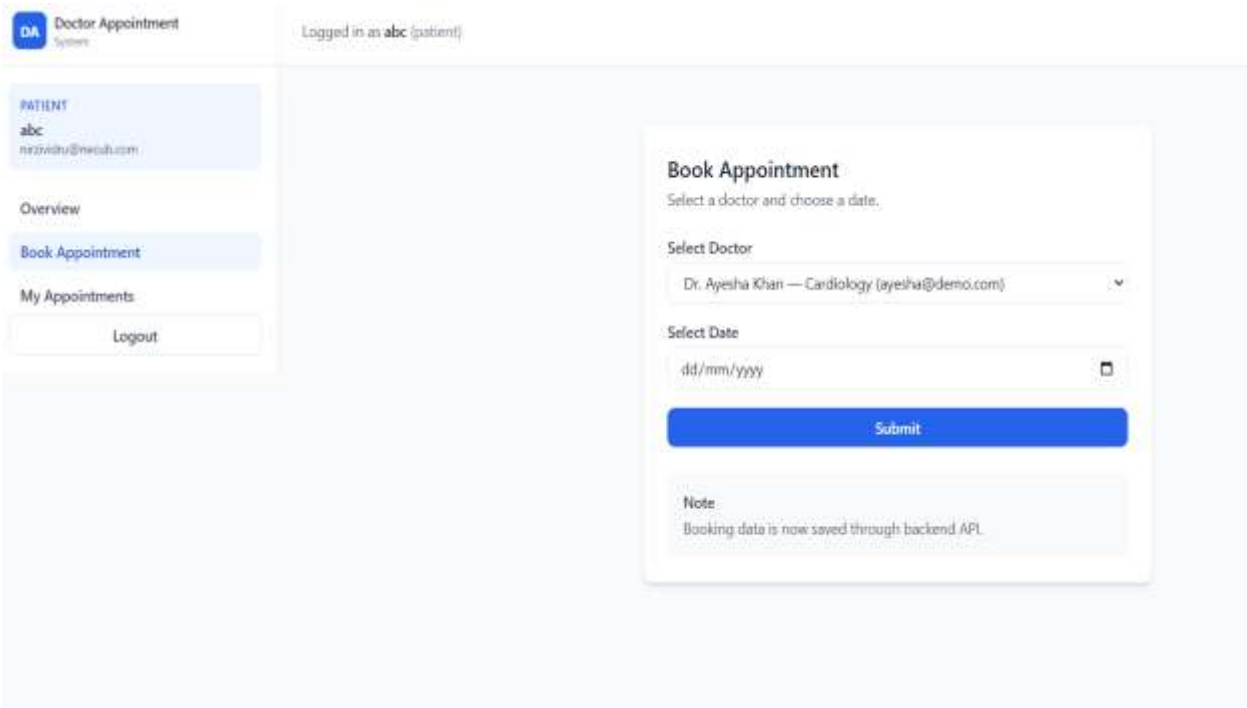


Fig: Patient book appointment page

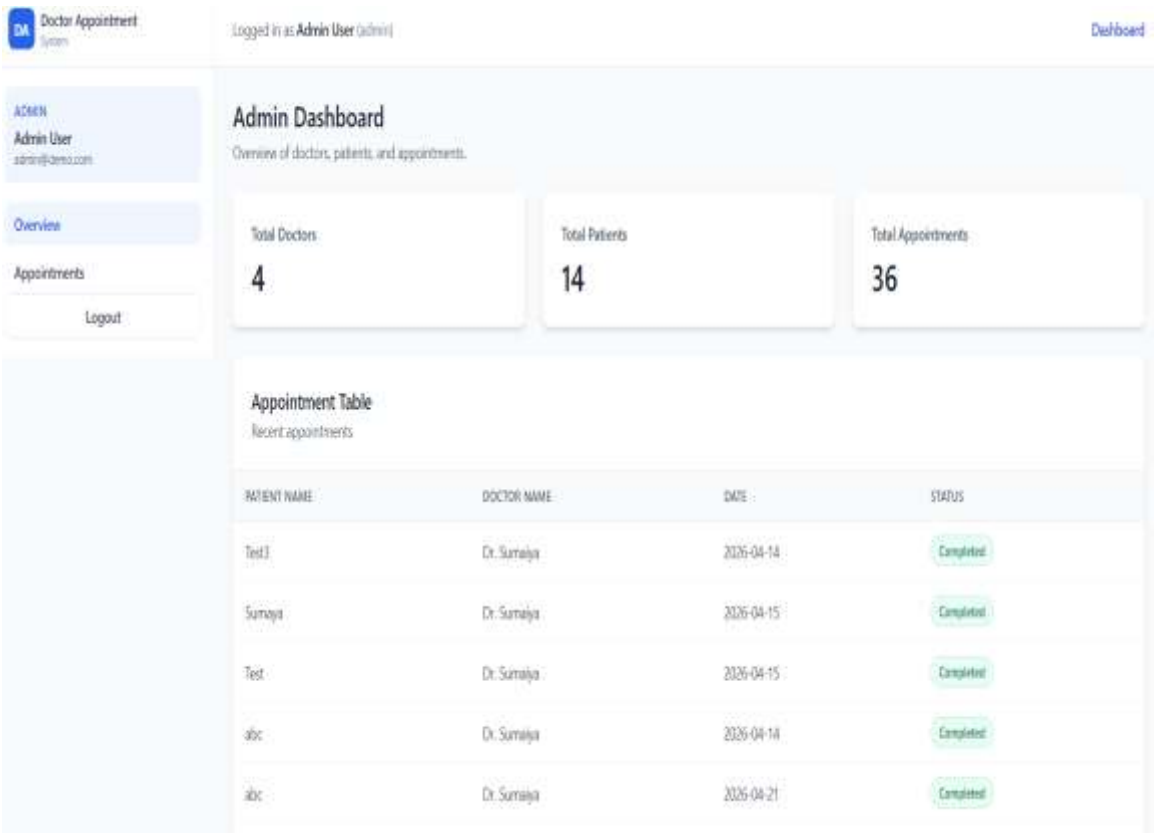


Fig: Admin Dashboard

14. Features

- **Full CRUD Operations:**
  The system allows users to perform all essential operations including creating, reading, updating, and deleting appointment data. Patients can book appointments, doctors can update statuses, and administrators can manage system records efficiently.
- **Secure Authentication System:**
  The application uses JWT-based authentication to ensure secure login and session management. Passwords are encrypted using hashing techniques, protecting user data from unauthorized access.
- **Responsive Design:**
  The system is fully responsive and optimized for different devices including mobile phones, tablets, and desktops. This ensures accessibility and a consistent user experience across all platforms.
- **Role-Based Access Control:**
  The system supports multiple user roles such as admin, doctor, and patient. Each role has specific permissions, ensuring proper access control and system security.
- **Protected Routes:**
  Sensitive pages and functionalities are protected and can only be accessed by authenticated users. Unauthorized users are restricted from accessing private data.
- **Appointment Status Management:**
  The system allows tracking and updating of appointment statuses such as Pending, Approved, Completed, and Cancelled. This helps maintain a clear workflow between patients and doctors.
- **Real-Time Data Updates:**
  Changes made to appointments or user data are instantly reflected in the system through API integration, ensuring up-to-date information for all users.
- **Email Notification System:**
  Automated email notifications are sent to users when appointments are created or updated. This enhances communication and keeps users informed about important changes.
- **Scalability and Performance:**
  Built using the MERN stack, the system is designed to handle multiple users efficiently and can be easily expanded with additional features in the future.

15. Implementation & Testing

15.1 Development Process:

The Doctor Appointment System was developed using a modular approach. The backend was built with Node.js and Express.js to create RESTful APIs for handling authentication, appointment management, and status updates. MongoDB was used as the database to store user and appointment data securely.

The frontend was developed using React.js to provide a dynamic and responsive user interface. It communicates with the backend through API calls and ensures smooth data flow. Authentication was implemented using JWT, and role-based access control was applied for security.

15.2 Testing Strategies:

- **Unit Testing:** Individual components such as forms and buttons were tested to ensure correct functionality.
- **Integration Testing:** Verified proper communication between frontend and backend APIs.
- **Security Testing:** Ensured that protected routes cannot be accessed without valid authentication.

16. Challenges & Solutions

Challenge 1: State Synchronization

- Problem: UI did not update immediately after updating appointment status or deletion.
- Solution: Implemented local state update in React to instantly reflect changes after a successful API response.

Challenge 2: Route Protection

- Problem: Unauthorized users could access protected pages via URL.
- Solution: Implemented Protected Routes using authentication (JWT) to restrict access and redirect unauthorized users to the login page.

Challenge 3: Email Not Sending in Production

- **Problem:** Email notifications worked in localhost but failed after deployment.
- **Solution:** Properly configured environment variables and SMTP settings (Gmail App Password, correct port) in the server.

Challenge 4: CORS Issues

- **Problem:** Frontend could not communicate with backend after deployment.
- **Solution:** Configured CORS in backend to allow requests from the deployed frontend domain.

17. Future Work

The current version of the Doctor Appointment System provides a strong foundation for managing appointments efficiently. However, there are several opportunities for future improvements to enhance functionality and user experience.

17.1 Online Payment Integration

In future updates, an online payment system can be added to allow patients to pay consultation fees during appointment booking. This will make the system more complete and reduce manual payment handling.

17.2 Video Consultation Feature

A video calling feature can be integrated to enable online doctor-patient consultations. This will be especially useful for remote users and emergency situations.

17.3 Appointment Reminder System

An automated reminder system (via email or SMS) can be added to notify patients before their scheduled appointments, reducing missed bookings.

17.3 Mobile Application Development

A dedicated mobile application (using React Native) can be developed to provide a better user experience and allow users to access the system more conveniently from smartphones.

17.5 Advanced Search & Filtering

Future versions can include advanced filtering options such as doctor ratings, specialization, and availability to help patients choose the best doctor

18. Conclusion

The Doctor Appointment System is a comprehensive solution designed to modernize the process of managing medical appointments. By utilizing the MERN stack, this project successfully replaces traditional manual scheduling with a fast, secure, and centralized web-based application. The integration of JWT authentication ensures that user data remains protected, while MongoDB provides a flexible and scalable database system.

Throughout the development of this project, emphasis was placed on creating a user-friendly interface that simplifies the process of booking and managing appointments. Patients can easily schedule appointments, and doctors can efficiently manage their schedules and update appointment statuses.

Additionally, the implementation of an email notification system improves communication by keeping users informed about appointment updates in real time. This enhances user experience and reduces confusion regarding appointment status.

In conclusion, the system improves overall efficiency, accuracy, and accessibility in healthcare appointment management. While the current version provides a strong foundation, its modular and scalable design allows for future enhancements such as online payments, video consultations, and mobile application support, making it a valuable long-term solution.

19. References

- In this project, various online resources, documentation, and tools were consulted to ensure the successful implementation of the system. The following sources were used as references:

- React Documentation: Official documentation for building the user interface and managing component states. (URL: https://react.dev)

- Node.js & Express.js Manuals: Used for structuring the backend API and handling server-side logic. (URL: https://expressjs.com)

- MongoDB Atlas Documentation: Referenced for database schema design and cloud hosting configurations. (URL: https://www.mongodb.com/docs)

- Firebase Authentication Guide: Followed for implementing secure user login and session management. (URL: https://firebase.google.com/docs/auth)

- Stack Overflow: Consulted for troubleshooting specific bugs and optimizing API performance.
- W3Schools & MDN Web Docs: Used for standardizing HTML5, CSS3, and JavaScript (ES6) coding practices.

- YouTube Tutorials: Various full-stack development tutorials for integrating MERN stack components.